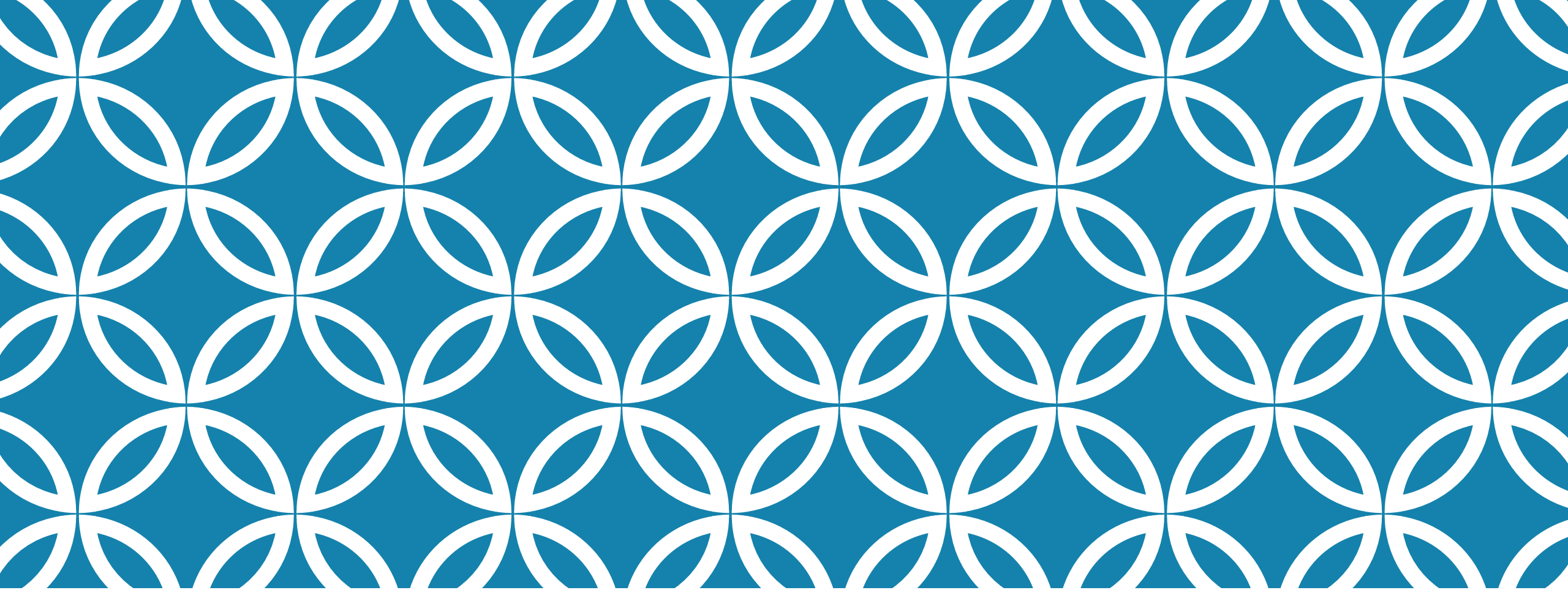
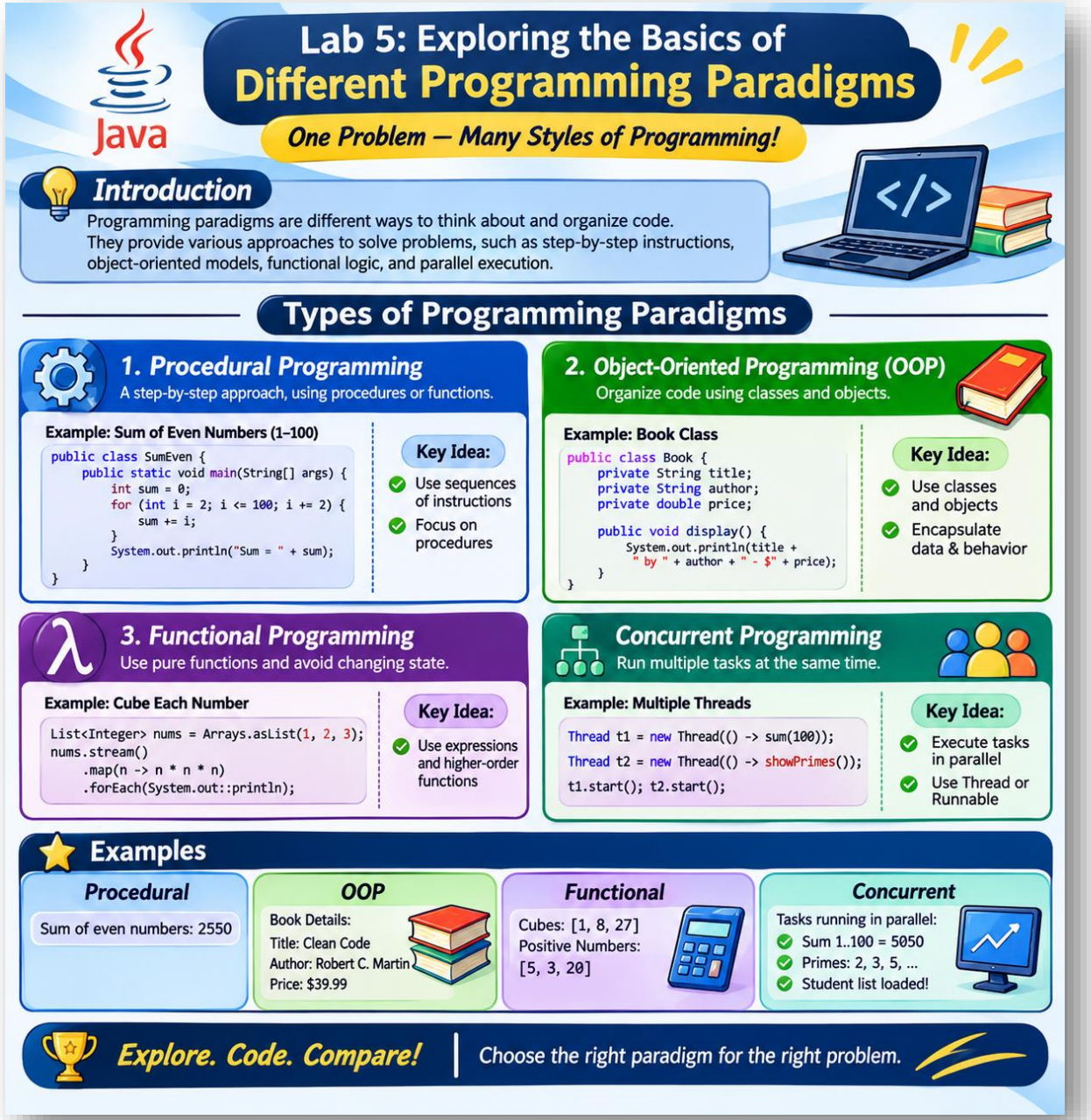


PROGRAMMING DESIGN AND IMPLEMENTATION (PDI)

Programming paradigms

OUTLINE

- **Introduction**
- **Types of programming paradigms**
  - Procedural programming
  - Object-oriented programming (OOP)
  - Functional programming
  - Concurrent programming
- **Examples**



INTRODUCTION



- **A programming paradigm** is a style or approach to programming that provides a way to structure and design software.
- Different paradigms are suited for solving different kinds of problems.

TYPES OF PROGRAMMING PARADIGMS

Four major programming paradigms:

- Procedural Programming,
- Object-Oriented Programming (OOP),
- Functional Programming, and
- Concurrent programming



PROCEDURAL PROGRAMMING

Procedural programming is based on the concept of procedures or routines, which are sets of instructions to be executed sequentially.

- Focuses on functions and procedures.
- Relies on a top-down approach.
- Uses variables, loops, and conditionals extensively

EXAMPLES:

Write a program to calculate the factorial of a number using procedural programming.

Write a program to find a highest score from an array containing scores of 100 students.

Write a program to ask a user for a number n. Display all numbers from 1 to n. Also find some statistics data including sum, average, max and min.

EXAMPLE



```
public class Factorial {
    public static int calculateFactorial(int n) {
        int result = 1;
        for (int i = 1; i <= n; i++) {
            result *= i;
        }
        return result;
    }

    public static void main(String[] args) {
        int number = 5;
        System.out.println("Factorial of " + number + " is: " + calculateFactorial(number));
    }
}
```

OBJECT-ORIENTED PROGRAMMING

- Object-oriented programming organizes code using objects and classes.
- It focuses on encapsulating data and behavior.

- **Class**: A blueprint for creating objects, containing fields (attributes) and methods (functions).
- **Object**: An instance of a class. A real item created from a class.

Data initialization and resource deletion

- **Constructor**: Special method called when an object is created. Used for initialization.
- **Destructor**: Cleans up resources (not explicit in Java, but handled by garbage collection).

Restricting access to attributes and methods

- **Public**: Accessible from anywhere.
- **Private**: Accessible only within the class.

OOP EXAMPLES:

Let's define a class **Car** with its basic attributes and methods.



```
class Car {
    String brand;
    String model;
    int year;

    void displayInfo() {
        System.out.println("Brand: " + brand + ", Model: " + model + ", Year: " + year);
    }
}

public class CarTest {
    public static void main(String[] args) {
        Car car = new Car();
        car.brand = "Toyota";
        car.model = "Corolla";
        car.year = 2020;
        car.displayInfo();
    }
}
```

OOP EXAMPLES:

Let's define a class **Student** with its basic attributes and methods.

```
class Student {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void displayInfo() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

public class StudentTest {
    public static void main(String[] args) {
                Student student = new Student("Alice", 22);
                student.displayInfo();
    }
}
```

OOP EXAMPLES:

Let's define a class **BankAccount** with its private attribute and public methods.

```
class BankAccount {
    private double balance;
    public BankAccount(double initialBalance) {
        balance = initialBalance;
    }
    public void deposit(double amount) {
        balance += amount;
    }

    public double getBalance() {
        return balance;
    }
}

public class BankAccountTest {
    public static void main(String[ ] args) {
        BankAccount account = new BankAccount(1000);
        account.deposit(500);
        System.out.println("Current Balance: " + account.getBalance());
    }
}
```

FUNCTIONAL PROGRAMMING (FP)

Functional programming treats computation as the evaluation of mathematical functions. It avoids changing state and mutable data.

- Emphasis on immutability.
- Use of higher-order functions.
- Avoidance of side effects.

Remark:

- **A mutable object** can be changed after it's created, and an immutable object can't.
- If you're defining your own class, you can make its objects immutable by making all fields final and private.
- Strings can be mutable or immutable depending on the language.

FUNCTIONAL PROGRAMMING

Key concepts

- **Functions as first-class citizens**
  You can store functions in variables, pass them to methods, and return them from methods.
- **Pure functions**
  - Same input → same output
  - No side effects (no modifying external variables, no I/O inside)
- **Immutability**
  Data does not change after creation.
- **Higher-order functions**
  Functions that accept other functions or return functions.
- **Declarative style**
  Focus on *what* to do, not *how* to do it.

FP EXAMPLE:

```java
import java.util.Arrays;
import java.util.List;

public class DP {
    public static void main(String[] args) {
        printSquare();
        System.out.println("----------------------");
        System.out.println("----------------------");
        displayEven();

    }

    public static void printSquare() {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
        numbers.stream().map(n -> n * n).forEach(System.out::println);


    }
    public static void displayEven(){
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
        numbers.stream().filter(n -> n % 2 == 0).forEach(System.out::println);
    }

}
```





```java
package lab5;

import java.util.Arrays;

public class FP {
    public static void main(String[] args) {
        Arrays.asList(1, 2, 3, 4, 5)
              .stream()
             // .filter(n -> n % 2 == 0)          // keep evens
              .map(n -> n * n)                    // square them
              .forEach(System.out::println);      // display
    }
}
```



Console ×

<terminated> FP [Java Application]

```
1
4
9
16
25
```

CONCURRENT PROGRAMMING (CP)

- **Concurrent programming (CP)** is a technique that enables multiple threads to execute simultaneously, improving application performance, especially on multi-core processors.
- Java provides a rich set of APIs and utilities to implement concurrency, primarily through the **java.util.concurrent** package.

CP Example: **thread** using inheritance

```
1  package lab4;
2  public class TestThread {
3      public static void main(String[] args) {
4          // TODO Auto-generated method stub
5
6          Thread th1 = new Thread(){
7              @Override
8              public void run() {
9                  for(int k=1; k<20; k++) {
10                     System.out.println(k);
11                 }
12             }
13         };
14         Thread th2 = new Thread(){
15             @Override
16             public void run() {
17                 System.out.println("Hi! I am in Thread2");
18                 System.out.println("DONE! in thread2");
19             }
20         };
21
22         th1.start();
23         th2.start();
24     }
25 }
```



```
Console ×
<terminated> TestThread [Java Application]
1
2
3
4
5
6
7
8
9
10
11
Hi! I am in Thread2
DONE! in thread2
12
13
14
15
16
17
18
19
```

```java
package lab5;

class MyThread extends Thread {
    @Override
    public void run() {
        System.out.println("Thread is running: " + Thread.currentThread().getName());
        for(int k=1; k<=4; k++) {
            System.out.println("Hi "+k);
        }
    }
}

class SimpleThread {
    public static void main(String[] args) {
        MyThread t = new MyThread(); // create thread object
        t.start();                   // start the thread

        for(int k=1; k<=4; k++) {
            System.out.println("Welcome "+k);
        }
    }
}
```

Create **MyThread** class extending from **Thread** (inheritance)



Console

<terminated> SimpleThread [Java Application] C:\User

```
Welcome 1
Welcome 2
Thread is running: Thread-0
Welcome 3
Welcome 4
Hi 1
Hi 2
Hi 3
Hi 4
```

Console

<terminated> SimpleThread [Java Application] C:\User

```
Welcome 1
Thread is running: Thread-0
Welcome 2
Welcome 3
Hi 1
Hi 2
Hi 3
Hi 4
Welcome 4
```

Console

<terminated> SimpleThread [Java Application] C:\Users\

```
Welcome 1
Welcome 2
Welcome 3
Welcome 4
Thread is running: Thread-0
Hi 1
Hi 2
Hi 3
Hi 4
```

Create a class **SimpleThread**.
Then run it in main. Hit run multiple times and observe on output

CP Example: **thread** using runnable task

```
public class SimpleRunnableExample {
    public static void main(String[ ] args) {
        // Create a Runnable task
        Runnable task = () -> {
            System.out.println("Thread is running: " + Thread.currentThread().getName());
        };

        // Create a Thread with the Runnable task
        Thread thread = new Thread(task);


        thread.start();
    }
}
```



```java
package lab5;
public class TestThreadCounter {
    private int count = 0;

    public synchronized void increment() {
        count++;
    }

    public int getCount() {
        return count;
    }

    public static void main(String[] args) throws InterruptedException {
        TestThreadCounter counter = new TestThreadCounter();

        Runnable task = () -> {
            for (int i = 0; i < 1000; i++) counter.increment();
        };

        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);

        t1.start(); t2.start();
        t1.join(); t2.join();

        System.out.println("Final count = " + counter.getCount());
    }
}
```

Faster computation by sharing task with different thread & using multiple CPU core

Task



Console ×

<terminated> TestThreadCounter [Java Application]

```
Final count = 2000
```

```java
package lab5;

import java.util.concurrent.CountDownLatch;

public class CountDownThread {
    public static void main(String[] args) throws InterruptedException {
        //CountDownLatch (waiting for tasks to finish)
        CountDownLatch key = new CountDownLatch(5);

        for (int k = 1; k<=5; k=k+1) {
            new Thread(() -> {
                System.out.println(Thread.currentThread().getName() + " done");
                key.countDown();
            }).start();
        }

        key.await(); // wait until all threads finish
        System.out.println("All tasks completed!");
    }
}
```



Console ×

<terminated> CountDownThread [Java Application]

```
Thread-0 done
Thread-4 done
Thread-3 done
Thread-2 done
Thread-1 done
All tasks completed!
```

Using CountDownLatch to wait until all threads finish before we move on.

Q&A

LAB 5

Programming paradigms

LAB05

OBJECTIVES: Explore the different types of programming paradigms.

- Procedural programming
- OOP
- Functional programming
- Concurrent programming

LAB5-EXERCISE 1

Write a procedural program to find the sum of even numbers from 1 to 100. Display result on screen.

LAB5-EXERCISE 2

Implement a calculator that supports basic arithmetic operations (+, -, *, /, sum from 1 to n) using a menu-driven approach.

LAB5-EXERCISE 3

Create a class **Book** with attributes like title, author, and price. Define 2 constructors (one constructor override the default and set default values to data while another constructor set info of a book).

Include a method to display the book's details.

LAB5-EXERCISE 4

Write a Java program to demonstrate a constructor by initializing attributes of an **Employee** class. Provide at least 5 attributes to describe an employee.

Use **private** attributes in a class and provide public getter and setter methods. Test your program in main.

LAB5-EXERCISE 5

Compare the execution of a factorial function implemented procedurally and using recursion. Let define input to this function as 3, 6, 10 and 50. What are the result output?

LAB5-EXERCISE 6

Design two classes **Rectangle** (methods to calculate area and perimeter), and Circle (methods to find surface and perimeter). Use encapsulation to manage width and height for Rectangle. Also use encapsulation to manage radius for the Circle. Test these classes in Main.

LAB5-EXERCISE 7

Write a program that can creates and runs multiple threads using **Thread** class.

Let's define 3 functions to:

a) find sum from 1 to n,

b) show all prime numbers in between 1 to 100,

c) read data from a CSV file of student list (name, ID, age, gender) and store it into an array list.

LAB5-EXERCISE 8

Redesign the problem in Exercise 7 by using Runnable task instead.

LAB5-EXERCISE 9

Write a functional program to:

a) find the cube of each number in a list and print the results

b) filter out positive numbers from a given list and print them.

c) sort data in a list.

```
List<Integer> numbers = Arrays.asList(5, 2, 9, -1, 3, -8, 20, -50);
```

Hint: Use sorted( ) method in the stream. numbers.stream( ).sorted( )

LAB5-EXERCISE 10

Read name of students from a CSV file then store in an array list.

Print out name of students in a sorted A-Z order using functional programming.

LAB5-EXERCISE 11

Find max and min in an array using functional programming.

```
int[ ] numbers = {5, 2, 9, 1, 3, 8};
```

Hint: To find max, you can use the class **IntStream** and method **max( ):**

```
IntStream.of(numbers).max();
```